

PRACTICAL WEEK 3

Task 1: Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution

CODE:

```
File: process.c
int main() {
    pid_t pid;
    printf("== Parent Process: Before fork() ==\n");
    printf("PID : %d\n", getpid());
    printf("PPID : %d\n", getppid());
    printf("State: Running\n");

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        // Child process
        printf("== Child Process ==\n");
        printf("PID : %d\n", getpid());
        printf("PPID : %d\n", getppid());
        printf("State: Running\n");

        sleep(3);

        printf("\n== Child Process: After sleep ==\n");
        printf("PID : %d\n", getpid());
        printf("PPID : %d\n", getppid());
        printf("State: Running\n");

        printf("Child process exiting...\n");
        wait(0);
    }
    else {
        // Parent process
        printf("== Parent Process: After fork() ==\n");
        printf("PID : %d\n", getpid());
        printf("PPID : %d\n", getppid());
        printf("Child PID: %d\n", pid);
        printf("State: Running\n");

        printf("\nParent is waiting for child...\n");
        printf("State: waiting\n");

        wait(NULL);

        printf("\n== Parent Process: After child terminates ==\n");
        printf("PID : %d\n", getpid());
        printf("PPID : %d\n", getppid());
        printf("State: Running\n");

        printf("Parent process exiting...\n");
    }

    return 0;
}
```

OUTPUT:

```
streenikhilreddy@MacBookPro ~ % nano process.c
streenikhilreddy@MacBookPro ~ % gcc process.c -o process
streenikhilreddy@MacBookPro ~ % ./process
=== Parent Process: Before fork() ===
PID : 1738
PPID : 1664
State: Running

=== Parent Process: After fork() ===
PID : 1738
PPID : 1664
Child PID: 1739
State: Running

Parent is waiting for child...
State: Waiting
=== Child Process ===
PID : 1739
PPID : 1738
State: Running

=== Child Process: After sleep ===
PID : 1739
PPID : 1738
State: Running
Child process exiting...

=== Parent Process: After child terminates ===
PID : 1738
PPID : 1664
State: Running
Parent process exiting...
streenikhilreddy@MacBookPro ~ %
```

Task-2: Using Linux terminal commands (uname, lscpu, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

```
Last login: Sat Aug 15 11:37:18 on tty000
streenikhilreddy@MacBookPro ~ % ps
PID TTY          TIME CMD
1787 tty000    0:00.03 -sh
streenikhilreddy@MacBookPro ~ %
```

[illegible]

```

Last login: Sat Aug 15 11:38:18 on tty000
zrenshihireny@MacBookPro ~ % open("sample.txt")
read()
write()
close()
function function function>
function function function> clear
zsh: number expected
zrenshihireny@MacBookPro ~ % open("sample.txt")
read()
write()
close()
function function function>

```

Command	Resource/Information	OS Concept
uname -a	Kernel and system information	Kernel abstraction
lscpu	CPU architecture and cores	CPU management
ps aux	Running processes	Process management
top	Real-time CPU/memory/process usage	Resource monitoring

Conclusion

The experiment demonstrates that the **operating system acts as an abstraction layer between applications and hardware**. The `uname` command provides information about the Linux kernel and system architecture, while `lscpu` reveals CPU resources managed by the OS. The `ps` command shows how the OS manages individual processes, and `top` provides real-time information about CPU, memory, and process utilization.

The OS abstracts **CPU through process scheduling, memory through virtual memory, storage through files and file systems, and I/O devices through device drivers and system calls**. Therefore, applications can use hardware resources without directly communicating with the physical hardware. This abstraction improves **security, resource sharing, portability, and efficient system management**.